

Pécsi Tudományegyetem
Műszaki és Informatikai Kar

Informatika és Villamos Intézet

Mérésadatgyűjtésre optimalizált fájlrendszer erőforráskritikus beágyazott rendszerekhez

Készítette: Vadász Péter Zalán

Elérhetőség: zalanvadasz@gmail.com

Belső konzulens: Dr. Várady Géza PhD

Beosztása: egyetemi docens

Pécs

2026

Tartalomjegyzék

1. Bevezetés és célkitűzés	3
2. A fájlrendszer alkalmazási környezete	5
2.1. Fájlrendszerekről általánosságban	5
2.1.1. A fájlrendszer definíciója	5
2.1.2. A fájlrendszerek típusai	5
2.2. Elméleti megkötések	10
2.2.1. Naplózás és írási sorrend	10
2.2.2. Összeomlásálló-konzisztencia invariánsok	11
2.2.3. ACID tulajdonságok	11
2.2.4. BASE modell	12
2.2.5. CAP-tétel (Brewer-tétel)	13
2.2.6. Idempotencia	14
2.2.7. Másolás íráskor (Copy-on-Write (CoW)) elv	15
2.2.8. Konzisztencia vs. teljesítmény kompromisszum	15
2.3. A fájlrendszer elsődleges felhasználási területei	16
2.3.1. CanSat	16
2.3.2. CubeSat	17
2.3.3. Beágyazott rendszerek	18
3. Fájlrendszerek elemzése a mérési-adatgyűjtési felhasználás szempontjából	19
3.1. F2FS	19
3.2. SPIFFS	19
3.3. YAFFS2 és JFFS2	20
3.4. LittleFS	20
4. Háttértárak működése	22
4.1. Mágneses elvű háttértárak (HDD)	22
4.2. Szilárdtest-meghajtók (SSD)	22
4.3. NAND és NOR flash memóriák	23
4.4. eMMC	23
4.5. SD-kártyák	23
5. Probléma megfogalmazása	24
6. ZINF funkcionális követelményei (Most Viable Product)	26
7. ZINF nem funkcionális követelményei (Not-MVP)	28

8. ZINF architektúrális felépítése	29
8.1. Logikai felépítés	29
8.1.1. Partíciók és tükrök	29
8.1.2. A tükrök belső struktúrája	30
8.2. Működési felépítés	33
8.2.1. Rétegek	33
8.2.2. Programozási nyelv	34
8.2.3. Fordítási környezet	34
8.3. A felhasználói interfész	35
8.3.1. A választott technológiai stack: Tauri és Svelte	36
8.3.2. A technológiai döntés indoklása	36
9. Fejlesztés menete / Történeti bemutatás	38
9.1. Interakció blokk alapú eszközökkel	38
9.2. Blokkok és szektorok	38
9.3. Visszaolvasás	38
9.4. Kód absztrakció	39
9.5. Validálás	39
9.6. Visszaállítás / javítás	39
9.7. Első alkalmazás valós környezetben	40
9.8. Illesztőprogram interfész	40
9.9. Konfigurálhatóság, felhasználóbarátság	40
9.10. Random írásvédelem	40
9.11. Utolsó szektormutató réteges védelme	41
10. A fájlrendszer tesztelése és validációja	42
10.1. Tesztelési módszertan és környezet	42
10.2. Teljesítménymérés és determinisztikusság	42
10.3. Hibatűrés és Integritás vizsgálata	43
10.3.1. Hibainjektálási szenáriók	43
10.3.2. Helyreállítás és Validáció	43
10.4. Valódi környezetben bekövetkező hibák	45
11. Összefoglaló	46
12. Köszönetnyilvánítás	47
13. Irodalomjegyzék	48

1 Bevezetés és célkitűzés

Bevezetés A kutatási és mérésadat-gyűjtési feladatok során hatalmas mennyiségű adatot kell rögzíteni, feldolgozni és hosszú távon tárolni olyan környezetekben, ahol az energia-ellátás, a hardvererőforrások és a háttértárak megbízhatósága erősen korlátozott. Ilyen rendszerek gyakran szélsőséges körülmények között működnek – például műholdakon, sarkvidéki berendezésekben vagy más autonóm mérőplatformokon – sokszor emberi felügyelet lehetősége nélkül.

Bizonyos környezetekben, például webalkalmazások fejlesztésekor, az optimalizálatlan kód és architektúra gyakran egyszerűen ellensúlyozható több erőforrás biztosításával, például egy erősebb szerverrel. Beágyazott rendszerekben azonban ez nem járható út: itt a szoftvernek kell alkalmazkodnia a szigorú korlátokhoz. Ezekben a rendszerekben elengedhetetlen a kód, az erőforrás-felhasználás és a háttértár-műveletek gondos optimalizálása.

Az adattárolásnál az elsődleges cél az adatintegritás, a determinisztikus működés és a hibákkal szembeni ellenállóképesség garantálása, miközben a rendelkezésre álló processzoridő, memória és háttértár mennyisége szigorúan limitált.

Hagyományos, általános célú fájlrendszerek alkalmazása ilyen környezetben nem szerencsés, mivel azok gyakran bonyolult metaadat-struktúrákra épülnek, nem determinisztikus időzítési viselkedést mutatnak, és nem tolerálják jól az áramkimaradást vagy a részleges írásokat. Ez indokolja egy olyan adattárolási megközelítés megtervezését, amely kifejezetten a kis erőforrású, energiaérzékeny és hibatűrő beágyazott platformok igényeihez igazodik.

Célkitűzés A rendszer célja egy olyan minimális, írás optimalizált, de teljes értékű adattárolási megoldás kialakítása, amely stabilan és megbízhatóan működik korlátozott környezetekben is. A tervezés során az alábbi fő célkitűzések kerültek meghatározásra:

- **Konzisztens adattárolás** még váratlan áramkimaradás, részleges írás vagy háttértár-hiba esetén is.
- **Determinisztikus időzítési viselkedés**, amely garantálja a valós idejű mérési és telemetria alkalmazásokban szükséges előrejelezhetőséget.
- **Alacsony erőforrásigény**, amely lehetővé teszi a használatát néhány kilobájt RAM-mal rendelkező mikrokontrolleres rendszereken.
- **Egyszerű és hordozható architektúra**, amely könnyen adaptálható különböző háttértárakhoz és platformokhoz.
- **Robusztus hibakezelés**, amely támogatja a blokkszintű hibadetektálást és szükség esetén redundáns tárolást.

-
- **Fejleszthető, hosszú távon fenntartható struktúra**, amely későbbi bővítésekhez, optimalizációhoz és új funkciók beépítéséhez is megfelelő alapot biztosít.

Ezen célkitűzések együttesen egy olyan erős, mégis egyszerű adattárolási megoldás létrehozását szolgálják, amely stabil alapot ad mérésadat-gyűjtéshez, telemetriai rendszerekhez és egyéb energia- és hibakritikus beágyazott alkalmazásokhoz.

2 A fájlrendszer alkalmazási környezete

2.1 Fájlrendszerekről általánosságban

2.1.1 A fájlrendszer definíciója

A fájlrendszer absztrakt adattípusok halmaza, amelyeket adatok tárolására, hierarchikus rendezésére, kezelésére, megtalálására illetve navigálásra, hozzáférésre, és visszakeresésére valósítottak meg.[1]

2.1.2 A fájlrendszerek típusai

Tárolási mód alapján Három típusát különböztetjük meg a fájlrendszereknek:

- Fájl,
- Objektum és
- Blokk

alapú tárolók.

Fájl tároló Ez a hagyományos értelemben vett fájlrendszer, amivel nap mint nap találkozunk azok a felhasználók akik NTFS-t (Windows), ext4-et (Linux) vagy APFS-t (MacOS) használnak. Ebben a fájlrendszerben az adatok úgy vannak tárolva, mint egy önmagában is teljesen értelmezhető információ. A fájl tartalmazza azt a mennyiségű információt magáról a tárolt adatról, metaadatot, ami szükséges ahhoz, hogy a fájl értelmezhető és használható legyen bármilyen külső extra információ nélkül. Azt hogy a fájl tartalmát miként kell értelmezni a fájl kiterjesztése fogja meghatározni (jpeg, wave, iso, docx, sql) Ahhoz egy ebben a konstrukcióban egy-egy fájlt megtaláljunk szükségünk van arra, hogy ismerjük az elérési útvonalát (path), hol helyezkedik el a gyökér (root) mappához képest. A fájlok hierarchikus módon vannak rendezve mappákon és almappákon keresztül. A fájlrendszerek nemcsak helyi háttértárakon jelennek meg, hanem szolgáltatásként is elérhetők. A modern felhőalapú infrastruktúrákban mindhárom tárolási modell – a fájl-, az objektum- és a blokkalapú tárolás – megtalálható, eltérő szolgáltatási szintekkel, teljesítménnyel és költségstruktúrával.

Előnyök

- Könnyű navigáció
- Intuitív minden felhasználó számára
- Egyszerű jogosultságkezelés

Hátrányok

- Nehéz skálázni
- Magas költségű bérlet

Felhasználási területek

- Hétköznapi felhasználás
- Kollaboráció
- Archiválás

Objektum tároló Ez egy lapos struktúrájú (flat structured) rendszer, amiben a fájlok kisebb diszkrét darabokra vannak bontva, objektumokra, és szétszórva helyezkednek el a tároló eszközön, egyetlen "mappában". Minden objektum egy önálló adattár, az értelmezéséhez szükséges összes információt tartalmazza. Az elérésükhöz ahelyett, hogy fájlnevet és elérési utat használnánk minden objektum rendelkezik egy egyedi azonosítóval. A fájlrendszerrel ellentétben szükséges az, hogy egy szabványosított interfészt használjunk a fájlok elérésére, egy API-t (Application Programming Interface).

Előnyök

- Nagy mennyiségű rendezetlen adat tárolására képes
- Kedvező gazdasági modell, elegendő csak a tárolt objektumok után fizetni
- Limit nélküli skálázhatóság és nagy tartósság
- Nagy metaadat kapacitás
- Könnyű benne keresni, a sok metaadat, tartalom és egyedi azonosító miatt

Hátrányok

- Limitált fájl záras
- Magas késleltetés
- Ha egy objektumot már létrehoztunk nem tudjuk módosítani, új példányt kell létrehozni. Újabb rendszerek már képesek verziózásra.

Felhasználási területek

- IoT adatkezelés
- Email archiválás
- Biztonsági mentések
- Mesterséges Intelligencia/Gépi tanulás

Blokk tároló A blokk tároló alapja, hogy az adatot kisebb fix méretű egységekre, blokkokra osztjuk, valamilyen megkülönböztető jelzővel ellátva. Minden blokk egy önálló adattár, az értelmezéséhez szükséges összes információt tartalmazza. Mikor a felhasználó elakarja érni az adatot, a rendszer megkeresi a blokkokat és újra összerakja őket egy egységgé. Míg a szervezetek egyre inkább az objektumtárolást alkalmazzák a nagy méretű, strukturálatlan adatokhoz és fájl tárolókat kollaborációs feladatokhoz, a blokk-tárolás továbbra is elengedhetetlen a konzisztens, alacsony késleltetésű hozzáférést igénylő nagy teljesítményű alkalmazások számára.

Előnyök

- Sebesség
- Megbízhatóság
- Könnyű kezelés

Hátrányok

- Kevés metaadat
- Alacsony kereshetőség
- Kedvezőtlen gazdasági modell

Felhasználási területek

- Virtuális gépek fájlrendszer-kötetek
- Enterprise alkalmazások
- Magas teljesítményt követelő kötetek, adatbázisok

Összehasonlításuk

- **Költség:** Mivel a blokk- és fájl tárolás költségei magasabbak, sok szervezet nagy adatmennyiségek tárolásához az objektumtárolást választja. Az objektumtárolás fogyasztásalapú fizetési modellje jelentős megtakarítást jelent a nagy méretű adat-tárolás esetében. Ez az előny ellentétben áll a blokk- és fájl tároláshoz szükséges előzetes hardverberuházásokkal.
- **Kezelési egyszerűség:** A metaadatok és a kereshetőség miatt az objektumtárolás a nagy adatmennyiségek tárolásának elsődleges választása. A fájl tárolás hierarchikus szervezési rendszerével inkább kisebb adatmennyiségek tárolására alkalmas.
- **Mennyiség:** A nagy adatmennyiségekkel rendelkező szervezetek gyakran az objektum- vagy blokk tárolást választják.
- **Visszakereshetőség:** Az adatok mindhárom tárolási típusból viszonylag könnyen visszakereshetők, bár a fájl- és objektumtárolás általában könnyebben hozzáférhető.
- **Metaadatok kezelése:** Az objektumtárolás általában jobban szolgálja a kiterjedt metaadatokkal rendelkező információkat, mint a fájl tárolás, amely csak alapvető metaadatokat tartalmaz.
- **Adatvédelem:** A tárolt adatokat meg kell védeni az adatvédelmi incidensek és a kiberbiztonsági fenyegetések ellen. A tárolás hatékony adatbiztonsági intézkedései közé tartozik a speciális engedélyek automatizálása, a titkosítás, az adatok maszkolása és az érzékeny fájlok szerkesztése.
- **Tárolási felhasználási esetek:** Minden tárolási típus különböző felhasználási esetekben és munkafolyamatokban a leghatékonyabb. A szervezetek a saját tárolási igényeik megértése révén kiválaszthatják azt a típust, amely a legtöbb tárolási felhasználási esetükhöz illeszkedik.

Tárolási stratégiák A megfelelő tárolási kombináció az aktuális feladat igényeitől és az adatok jellemzőitől függ. A tárolási architektúra tervezésekor figyelembe kell venni az adatmennyiséget, a teljesítménykövetelményeket, a költségvetési korlátokat és a növekedési előrejelzéseket.

- Az objektumtárolás a legalkalmasabb strukturálatlan adatok, generatív MI-munkafolyamatok és MI/Gépitanulás-alkalmazások számára, amelyek alacsony költség mellett hatalmas méretezhetőséget igényelnek.
- A fájl tárolás egy ismerős, hierarchikus szervezést biztosít, amely ideális együttműködéshez, dokumentumkezeléshez és kisebb léptékű műveletekhez.

-
- A blokk tárolás nagy teljesítményt nyújt adatbázisok, virtuális gépek és olyan alkalmazások számára, amelyek konzisztens, alacsony késleltetésű hozzáférést igényelnek.

[2][3][4]

2.2 Elméleti megkötések

2.2.1 Naplózás és írási sorrend

Mi az a naplózás (journaling)? Naplózás = a fájlrendszer a lemezre írandó változásokat először egy *napló-fájlba* (journal) írja, majd csak azután tekinti „véglegesnek”, hogy ezek a naplóbejegyzések (journal entries) a lemezre kerültek és véglegesítődtek.

- összeomlás esetén (áramkimaradás, kernel pánik) a napló újrajátszásával visszaállítható egy konzisztens állapot,
- ne maradjanak félkész struktúrák (pl. félig létrehozott inode-ok, könyvtárbejegyzések).

Négy fő naplózási mód van az ext3/ext4-ben:

1. **Csak-Metaadat (Metadata-only):** Csak a metaadat íródik a naplóba.
2. **Teljes adat + metaadat (Full Data + Metadata):** Az összes napló és metaadat is kiíródik a naplóba.
3. **Sorba rendezett (Ordered):** csak a metaadat kerül a journalba, de a rendszer garantálja, hogy az adatblokkokat *a metadata véglegesítés előtt* írja ki.
4. **Visszaírás (Writeback):** csak a metaadat kerül a naplóba, *nincs garantálva*, hogy az adatblokkok véglegesítés előtt kikerüljenek; gyorsabb lehet, de gyengébb adatkonzisztencia-garanciát ad.

Írási sorrend Ha a fájlrendszer nem definiál egyértelmű írási sorrendet *write order* szabályt az adat- és metaadat-blokkokra, akkor nem tud összeomlásálló-konzisztenciát (crash-consistency) garantálni: áramkimaradás után olyan állapotba kerülhet, ahol léteznek metaadatok olyan adatblokkokra hivatkozva, amelyek valójában soha nem kerültek kiírásra.

Rendezett módban:

- előbb az adatblokkok kerülnek kiírásra a fő fájlrendszer-területre;
- *utána* véglegesítődik a metaadat a naplóban;
- így ha összeomlás következik be, nem lehet olyan helyzet, hogy a metaadat már hivatkozik egy olyan adatblokkra, ami még nincs a lemezen.

Visszaírás módban:

- ez a garancia hiányzik;

-
- előfordulhat, hogy a metaadat már arra mutat, hogy „itt az új adat”, de maga az adat még a régi (vagy random szemét) a lemezen.

[5][6]

2.2.2 Összeomlásálló-konzisztencia invariánsok

Invariáns Az invariáns egy olyan tulajdonság a rendszer állapotáról, ami *mindig igaz kell hogy legyen*, ha a rendszer nem sérült.

- Minden könyvtárbejegyzés egy érvényes inode-ra mutat.
- Egy blokkról vagy azt mondjuk, hogy szabad, vagy hogy foglalt, de a kettő egyszerre nem lehet.
- Minden inode vagy elérhető egy útvonalon (könyvtár fán keresztül), vagy már töröltnek tekintjük.

Összeomlásálló-konzisztencia mint invariáns-megőrzés Egy fájlrendszer összeomlásálló-konzisztens, ha bármely ponton bekövetkező hiba (pl. áramkimaradás) után a napló újrajátszása vagy a fájlrendszer ellenőrzés (filesystem check = fsck) futtatása olyan állapotba hozza a rendszert, ahol a fájlrendszer invariánsai ismét teljesülnek. Ez nem feltétlenül jelenti azt, hogy *nem veszünk adatot*:

- lehetséges, hogy az épp íródó adatok elvesznek,
- de nem maradhat például olyan inode, ami árva (nincs rá hivatkozó könyvtárbejegyzés),
- és nem lehet olyan blokk ami egyszerre szabad is meg foglalt is.

Fájlrendszereszközök: naplózás és fsck Összeomlásálló-konzisztencia fenntartásában két fő mechanizmus szokott segíteni:

- **Naplózás:** előre feljegyzi, mit fog csinálni; hiba után a napló újrajátszásával állítja helyre az invariánsokat.
- **fsck:** utólag végignézi a struktúrákat, és egyenként javítja az invariáns-sértéseket (árva inode, rossz link-szám, stb.).

[7]

2.2.3 ACID tulajdonságok

Eredete Az ACID eredetileg adatbázis-tranzakciókhoz lett megfogalmazva, de ugyanúgy alkalmazható fájlrendszer-szintű tranzakciókra.

ACID = Atomicity, Consistency, Isolation, Durability .

- **Atomicity (atomosság):** a tranzakció vagy *teljes egészében* megtörténik, vagy *egyetlen része* sem.
- **Consistency (konzisztencia):** a tranzakció a rendszer invariánsait megőrizve viszi át az állapotot egyik érvényes állapotból a másikba.
- **Isolation (elszigeteltség):** párhuzamos tranzakciók egymástól úgy „elválasztva” futnak, mintha egymás után, sorosan hajtódnának végre.
- **Durability (tartósság):** ha a tranzakció lezárult (commit-olt), akkor az eredménye *tartós*: áramkimaradás után is úgy látjuk, mintha „beleégett” volna a lemezbe.

ACID fájlrendszerekben:

- Naplózó (journaling) fájlrendszerek (ext3, ext4, NTFS, stb.) megkísérlik, hogy egy-egy komplex művelet (pl. sok blokkból álló fájl létrehozása, inode frissítés, directory bejegyzés módosítása) *ACID-szerűen* történjen:
 - a naplóban (journal) rögzítik, mit fognak írni;
 - amikor a tranzakció lezárult véglegesítőjelzés (commit) mutatja, hogy a tranzakció sikeres volt-e vagy sem;
 - amennyiben hiba történt a napló újrajátszásával állítják helyre a konzisztens állapotot.
- Nem minden fájlrendszer ad teljes értékű ACID-garanciát, de:
 - az *Atomosság* és a *Tartósság* részben a naplózás módjától függ (teljes adat + metaadat, sorba rendezett, visszaírás),
 - a *Konzisztencia* sokszor abból áll, hogy bizonyos invariánsokat sosem sértenek meg (pl. nem lesz fél inode, árva könyvtárbejegyzés),
 - az *Elszigeteltség* pedig fájlrendszer-szinten jellemzően zárolásokkal (lockokkal), vagy egyszerűbb: „nincs párhuzamosság ugyanarra a fájlra” elvvel valósul meg.

[8]

2.2.4 BASE modell

BASE = Basically Available, Soft State, Eventual Consistency .

Basically Available (Alapvetően Elérhető) A rendszer alapvetően elérhető: *szinte mindig* válaszol, még akkor is, ha a válasz nem feltétlenül a legfrissebb verzió.

Soft State (Puha Állapot) A rendszer állapota puha, változhat *akkor is*, ha épp senki nem ír bele (pl. replikációk másolgatják egymás közt az adatokat).

Eventual Consistency (Fokozatos Konzisztencia) Ha egy darabig nem történik új módosítás, akkor a rendszer *előbb-utóbb konvergál* egy olyan állapotra, ahol minden csomópont ugyanazt az adatot látja.

Eredete A BASE modell az ACID modell ellentéte amire azért volt szükség, mert elosztott, nagy léptékű rendszerekben (pl. felhős adatbázisok, elosztott fájlrendszerek) gyakran nem lehet egyszerre:

- nagyon gyorsnak és mindig elérhetőnek lenni,
- miközben minden pillanatban globálisan konzisztens adatot adunk.

Ezért született meg a **BASE** szemlélet, ami inkább a *rendkívüli elérhetőséget és rugalmasságot* helyezi előtérbe, miközben elfogadja, hogy a konzisztencia csak *idővel* áll helyre.

Elosztott fájlrendszerek Az elosztott fájlrendszerek (MongoDB, Redis, Cassandra, stb.) gyakran BASE jellegűek:

- több replika tárolja ugyanazt a fájlt különböző gépeken;
- hálózati hiba esetén egyes replikák lemaradhatnak;
- a rendszer mégis „alapvetően elérhető” akar maradni: inkább ad egy kicsit régebbi verziót, minthogy ne adjon semmit;
- idővel (replikációs protokollok segítségével) a replikák újra összehangolódnak, ekkor valósul meg a tényleges konzisztencia.

[9]

2.2.5 CAP-tétel (Brewer-tétel)

Eredete A CAP-tétel az elosztott rendszerek egyik legismertebb elméleti eredménye. Röviden: *egy elosztott adatkezelő rendszer nem tudja egyszerre garantálni* a következő három tulajdonságot hálózati szétválás (partition) mellett:

- **C - Consistency (konzisztencia):** minden olvasás a *legfrissebb* írást látja (vagy hibát kap),
- **A - Availability (elérhetőség):** minden kérésre érkezik válasz (nincs olyan, hogy „nem tudom”),
- **P - Partition Tolerance (partíció tolerancia):** hálózati szakadás, csomagvesztés esetén is működőképes marad a rendszer.

Szabály: *Egy elosztott adatkezelő rendszerben hálózati partíciók jelenléte esetén a rendszer egyszerre legfeljebb kettőt garantálhat a fenti három tulajdonság közül.*

- ha P -t muszáj tartani (, a gyakorlatban igen),
- akkor választanunk kell C és A között: vagy inkább konzisztens ($\Rightarrow CP$), vagy inkább elérhető ($\Rightarrow AP$) rendszert építünk.

CAP tétel összefoglalva

„A CAP-tétel szerint elosztott környezetben, hálózati szakadás esetén egy adatkezelő rendszernek választania kell a konzisztencia és az elérhetőség között; a hálózati partíciókkal szembeni tűrést csak az egyik rovására tudja biztosítani.”

Fájlrendszerek szempontjából ez azt jelenti, hogy elosztott fájlrendszer tervezésekor *nem lehet egyszerre mindenben olyan kényelmes, mint egy lokális ext4 partíció*: vagy időnként hibát kap az alkalmazás, vagy néha nem teljesen friss adatot. [10]

2.2.6 Idempotencia

Fogalma Egy művelet **idempotens**, ha többszöri végrehajtása *ugyanazt az állapotot* eredményezi, mint egyszeri végrehajtása.

- Kapcsoló: ha a lámpa már fel van kapcsolva, akkor a „kapcsold fel” utasítás még egyszer végrehajtva ugyanazt az állapotot adja (a lámpa fel van kapcsolva).
- Beállítás: „állítsd a hangerőt 50%-ra” — ha már 50%, akkor még egyszer futtatva nem változik semmi.

Idempotencia fájlrendszerben Fájlrendszer-műveletek, amelyek tipikusan idempotensek vagy közel azok:

- `mkdir /valami` akkor is „ugyanazt” az eredményt adja, ha véletlenül kétszer fut le (feltéve, hogy közben senki nem törli a könyvtárat);
- `unlink` (törlés): ha már töröltük a fájlt, egy újabb `unlink` ugyanarra a névre lényegében ugyanoda vezet (a név már nincs ott).

Miért fontos ez az összeomlás utáni helyreállítás (crash recovery) és a naplózás során?

- Bizonyos műveleteket a rendszer *újra lefuttat*, mert nem biztos benne, hogy az előző futás során sikerült-e teljesen végrehajtani őket (áramkimaradás stb.).

-
- Ha a naplózás által újrarátszott lépések *idempotensek* (vagy legalábbis úgy lettek megtervezve, hogy többszöri lefutásuk ne rontsák el az állapotot), akkor az összeomlás utáni visszaállítás során garantálható, hogy a helyreállított állapot konzisztens marad.

[11]

2.2.7 Másolás íráskor (Copy-on-Write (CoW)) elv

Alapötlet Copy-on-Write (CoW) rendszerekben (pl. ZFS, Btrfs, APFS) *soha nem írjuk felül* a meglévő adatblokkokat. Ehelyett:

1. ha módosítani akarunk valamit, új blokkot foglalunk, oda írjuk az új adatot,
2. majd a metaadatokat (pl. pointert) úgy frissítjük, hogy az az új blokkra mutasson.

Összefoglalva:

CoW fájlrendszerben nincsen olyan művelet, amely egy létező blokk tartalmát felülírja. Minden módosítás új blokkba íródik, és az állapotváltozás lényege a mutatók átállítása.

Következmények:

- **Erős Összeomlásálló-konzisztencia:** ha a mutató-csere előtt történik az összeomlás, akkor még a régi, teljesen konzisztens állapotot látjuk; az új blokk „árván” maradhat, de hibamentes.
- **Pillanatképek (Snapshot):** régi mutatóstruktúrák megőrzésével könnyen készíthetünk konzisztens pillanatképeket.
- **Ellenőrzőösszeg-alapú (checksum) hibajavítás:** Sok CoW fájlrendszer blokk-szintű ellenőrzőösszeget használ, így ha egy replika sérült, egy másikból helyreállítható.

[12]

2.2.8 Konzisztencia vs. teljesítmény kompromisszum

Három cél, amiből kettőt könnyű egyszerre elérni Fájlrendszer-tervezésnél megjelenhet egy CAP-tétel szerű kompromisszum a nem elosztott (helyi) rendszerekre is:

- **Erős konzisztencia** (minden művelet után azonnali stabil állapot),
- **Magas teljesítmény** (minimális írási késleltetés, nagy átviteli sebesség),
- **Alacsony írás-amplifikáció** (ne kelljen ugyanazt az adatot sokszor lemezre írni).

Összefoglalva:

Egy gyakorlatban megvalósított fájlrendszerben általában nem lehet egyszerre maximális konzisztenciát, maximális teljesítményt és minimális írási költséget elérni. A tervezőnek választania kell, melyik szempontból enged.

- **Erős konzisztencia + alacsony írás-amplifikáció:** egyszerűbb, naplózás nélküli fájlrendszerek, de gyengébb összeomlásálló-konzisztenciával.
- **Erős konzisztencia + magas teljesítmény:** agresszív gyors elérésű (illékony) memóriában való tárolás (cache), de áramszünet esetén nagyobb adatvesztés kockázat.
- **Összeomlásálló-konzisztencia + extra írás:** naplózás / CoW, ahol a konzisztencia érdekében többet írunk (napló + fő terület).

2.3 A fájlrendszer elsődleges felhasználási területei

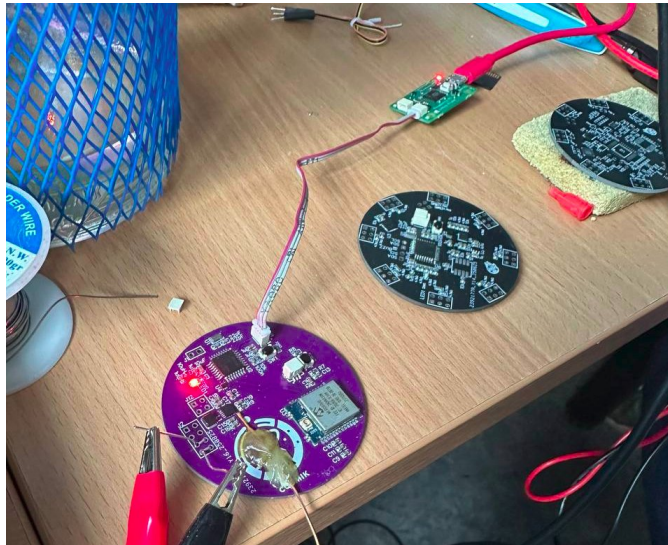
2.3.1 CanSat

A CanSat egy rendkívül kisméretű, oktatási célokra fejlesztett műholdmodell, amely nem hagyja el a Föld légkörét. A formai korlátokat tekintve egy 66 mm átmérőjű és 115 mm magas, legfeljebb 350 g tömegű hengerbe kell minden fedélzeti rendszert integrálni. Tipikusan szuborbitális rakéták vagy meteorológiai ballonok juttatják a magasléggörbe, ahol valós mérési és telemetriai feladatokat kell teljesítenie. Bár földközeli környezetben működik, a tervezés és a rendszerintegráció szempontjából teljes mértékben egy valódi műhold felépítését és működési folyamatait tükrözi, így kiváló gyakorlati alapot biztosít az űripari mérnöki kompetenciák elsajátításához.

A Juhász Jenő Szakkollégium CosMIK hallgatói csoportja 2025-ben fejlesztette ki a Mimike-II Rev-I CanSat rendszert. Ennek a csoportnak én vagyok hallgatói oldalról a vezetője, a Mimike-II-ön pedig, mint vezető szoftverfejlesztő és projektmenedzser dolgoztam. A projekt egy négy darab, egymásra épülő nyomtatott áramkörből álló moduláris fedélzeti egységet valósít meg, amely kifejezetten tanulási és kutatási célú, valós repülési körülményeket szimuláló mérésadat-gyűjtésre készült. A rendszer célja a magas megbízhatóságú szenzoradat-rögzítés, CanSat fő rendszerek szimulációja, saját fejlesztésű beágyazott szoftver, hardver és fájlrendszer (ZINF) tesztelése, valamint a CanSat- és CubeSat-szintű fedélzeti architektúrák gyakorlati elsajátítása. A Rev-I fejlesztési ciklusa sikeresen lezárult, a rendszer minden fő komponense – a fedélzeti elektronika, a szenzor-fal és a kommunikációs alrendszer – működött, és igazolta a tervezési elvek helyességét. A Juhász Jenő Szakkollégium CosMIK hallgatói csoportja a Rev-I tapasztalatai alapján tovább is lépett: jelenleg már a Rev-II változat kidolgozása zajlik.

A Rev-II célja a megbízhatóság növelése, a fedélzeti architektúra finomhangolása, a korábbi Rev-I során azonosított gyengeségek javítása, valamint új funkciók beépítése. A

fejlesztések között szerepel a teljesítményoptimalizálás, a kommunikációs és adatkezelési protokollok továbbfejlesztése, valamint a hosszabb küldetésekhez szükséges robusztusabb elektronikai kialakítás.



1. ábra. Juhász Jenő Szakkollégium által készített Mimike-II Rev-I CanSat
Forrás: saját kép

2.3.2 CubeSat

A CubeSat-ek már teljes értékű, kisméretű műholdak, amelyeket valós tudományos vagy technológiai kísérletek végrehajtására terveznek az űrben. Alapegységük a 10×10×10 cm-es kocka (1U), amelyből több egység összeillesztésével hoznak létre nagyobb rendszereket (2U, 3U, 6U vagy akár 12U konfigurációk). A miniatürizálás és elektronikai fejlődés miatti költséghatékonyságuknak és szabványosított felépítésüknek köszönhetően ma már a felsőoktatás, a kutatóintézetek és az ipar számára is széles körben elérhető platformot jelentenek új megoldások tesztelésére, földmegfigyelésre, kommunikációs technológiák kipróbálására vagy akár mélyűri missziók előkészítésére.



2. ábra. European Space Agency, ESEC-Galaxia Education Center, ESA Academy Training Week 2025, Theia ESAT CubeSat
Forrás: ESA Academy Training Week 2025, saját kép

2.3.3 Beágyazott rendszerek

Bár a fájlrendszer elsődlegesen CanSat és CubeSat környezetben való alkalmazásra készült, felépítése általánosan is alkalmazható bármilyen érzékelés- és mérésadatgyűjtés-központú beágyazott eszközben. A kompakt, energiahatékony, hibamentes adattárolást előtérbe helyező struktúra ideálissá teszi olyan területeken is, mint:

- ipari szenzorhálózatok és IoT platformok,
- környezeti monitoring állomások,
- autonóm robotikai rendszerek,
- távoli vagy nehezen hozzáférhető helyszíneken végzett adatgyűjtés,
- laboratóriumi mérőműszerek beépített adatnaplózása.

Az általam fejlesztett fájlrendszer tehát nem csupán oktatási vagy űrkutatási célokra alkalmas, hanem bármely olyan mérnöki feladatra, ahol nagy megbízhatóságú, kis erőforrásigényű, folyamatos adatnaplózásra optimalizált tárolómegoldásra van szükség.

3 Fájlrendszerek elemzése a mérési-adatgyűjtési felhasználás szempontjából

Az alábbi fejezet célja, hogy áttekintse a beágyazott rendszerekben elterjedt fájlrendszereket abból a szempontból, mennyire alkalmasak telemetriai, folyamatos mérési adatgyűjtési feladatokra, ahol fontos a determinisztikus működés, az áramkimaradás-tűrés, az alacsony erőforrásigény és a kiszámítható írási késleltetés.

3.1 F2FS

Az F2FS egy modern, naplóalapú (log-structured) fájlrendszer, amelyet kifejezetten FTL-es eszközökhöz (eMMC, SD, SSD) fejlesztettek ki.

Előnyök

- Kiváló teljesítmény FTL-alapú eszközökön.
- Naplóalapú kialakítás (szekvenciális íráshoz jó).
- Naplózás + több szintű metaadat a nagy kapacitáshoz.

Korlátok

- Nagy kódbázis és kernel-dependencia (Linux szükséges).
- Jelentős RAM és CPU igény, MCU-kon nem használható.
- Általános fájlrendszer, nem célzott telemetriai adatrögzítésre.
- Túlzottan NAND flash alapú tárhelyre tervezve.

[13]

3.2 SPIFFS

A SPIFFS egy egyszerű, egyfájlos logikai struktúrát használó fájlrendszer, amely kifejezetten SPI flash memóriákhoz készült. Működésének kulcseleme a garbage collection (GC).

Előnyök

- Kis méretű kód és viszonylag egyszerű implementáció.
- Tipikusan egyetlen logikai fájl kezel.
- Flash-specifikus optimalizációk.

Korlátok

- Szemétgyűjtés (GC) miatt telítettség közelében megnőhet az írási késleltetés.
- A GC miatt az írás nem determinisztikus.
- Nincs redundáns tárolás (csak CRC).
- Csak flash-re érhető el.

[14]

3.3 YAFFS2 és JFFS2

A YAFFS2 és a JFFS2 klasszikus, nyers NAND flashhez készült fájlrendszerek. A Linux rendszerekben elterjedtek, de mikrokontrollereken önmagukban ritkán használhatóak.

Előnyök

- Fejlett hibatűrés, ellenőrzőpont-rendszerek (checkpoint).
- Robusztus működés nagyobb méretű, nyers NAND tárolókon.
- Logikai integritásra optimalizált algoritmusok.

Korlátok

- Jelentős RAM- és CPU-igény.
- Összetett struktúrák, kernel-szintű támogatás szükséges.
- Tipikusan Linux alatt használható.

[15]

3.4 LittleFS

A LittleFS egy kompakt, modern, CoW elvű flash-fájlrendszer, amelyet mikrokontrollerekhez fejlesztettek ki. A struktúra fa-alapú metaadatokra épül, és erősen fókuszál a robusztus működésre.

Előnyök

- Beépített hiba elleni mechanizmus: CoW miatt áramkimaradás esetén is konzisztens marad.
- Alacsony memóriaigény, MCU-khoz optimalizálva.
- Beépített elhasználódás elleni védelem (wear-leveling).

Korlátok

- CoW miatt a tényleges írási költség nagy.
- A metaadat-struktúra viszonylag összetett.
- Több fájl és könyvtárat kezel, ami felesleges lehet telemetriai környezetben.

[16]



3. ábra. littlefs (bal) és YAFFS (jobb) logója

Forrás: <https://yaffs.net/yaffs-2-specification/>,
<https://avatars.githubusercontent.com/u/60944974?s=48&v=4>

4 Háttértárak működése

A beágyazott és mérésadat-gyűjtő rendszerekben különböző háttértár-technológiák találhatók, amelyek működési elve alapvetően meghatározza a rájuk épített fájlrendszer tervezését, teljesítményét és hibátűrését. Mivel sok fajta háttértár létezik, de nem lehet egyiket sem mindenhol felhasználni, mindig a feladatnak megfelelőt kell kiválasztani. Ezért szükséges minél több működésű háttértár ismerete és illesztőprogramjuk implementációja saját fájlrendszerembe, hogy feladatnak megfelelő háttértárat lehessen mindig alkalmazni.

4.1 Mágneses elvű háttértárak (HDD)

A hagyományos merevlemezek (HDD – Hard Disk Drive) mágneses bevonatú forgó lemezeket alkalmaznak. Az adatok olvasása és írása mechanikus fej segítségével történik.

Működési elv

A HDD-k jellegzetessége a mechanikus működésből fakadó változó hozzáférési idő (seek time + forgási késleltetés). Íráskor a mágneses területet közvetlenül módosítja a fej (in-place update).

Előnyök és Hátrányok

Nagy kapacitás és alacsony ár, determinisztikusabb írás. Ugyanakkor **beágyazott környezetben (pl. CanSat) kizárt**, mivel érzékeny a rezgésre, gyorsulásra és nagy az áramfelvétele.



4. ábra. HDD meghajtó

4.2 Szilárdtest-meghajtók (SSD)

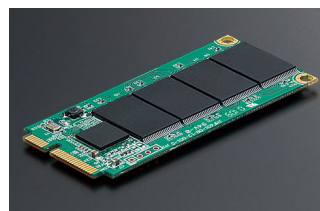
Az SSD-k NAND flash memóriára épülő, mechanikus alkatrész nélküli adattárolók, amelyek fejlett vezérlőchipet, DRAM gyorsítótárat és komplex firmware-t használnak.

Működési elv

NAND cellák blokk-szintű törlése és lap-szintű írása. Az FTL (Flash Translation Layer) végzi a logikai-fizikai címfordítást.

Determinisztikus tényezők

Az SSD teljesítménye ingadozó. A szeméthyűjtés (GC), az elhasználódás elleni védelem és a cache ürítése miatt egy írási művelet ideje váratlanul megnőhet (akár több tíz ms-ra).



5. ábra. SSD meghajtó

4.3 NAND és NOR flash memóriák

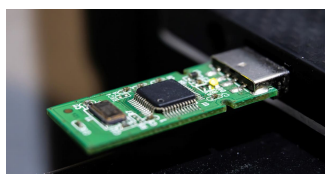
A nyers flash memóriák közvetlenül MCU-val is kezelhetők, vezérlőlogika nélkül.

NOR flash

Gyors, tetszőleges elérésű olvasás (XIP), kiváló kód/firmware tárolásra. Írása és törlése lassú, kapacitása kicsi.

NAND flash

Nagy kapacitás, alacsony költség. Csak blokk-szintű törlés, lap-szintű írás. Magasabb hibaarány miatt ECC és Bad Block Management szükséges a fájlrendszer szintjén. Jól skálázható.



6. ábra. Flash memória

4.4 eMMC

Az eMMC (embedded MultiMediaCard) a NAND flash memóriát és a vezérlőt egyetlen tokozásba foglalja.

Előnyök

Stabilabb és kiszámíthatóbb, mint az SD-kártya. Integrált FTL-t, wear-levelinget és hibakezelést tartalmaz. Ipari eszközökben elterjedt.

Hátrányok telemetriában

A belső működés gyártóspecifikus (black box). A háttérfolyamatok itt is okozhatnak váratlan késleltetést, így a valósidejű determinizmus nem garantált.



7. ábra. Multi Média Kártya

4.5 SD-kártyák

Az SD-kártya a mikrovezérlős környezetek leggyakoribb, de legkevésbé determinisztikus adattárolója. Belső FTL-je és cache-mechanizmusai teljesen el vannak rejtve a fejlesztő elől.

Protokoll és működés

SPI vagy natív SD módban használhatók. A belső vezérlő automatikusan végzi a blokkmenedzsmentet, ami felhasználóbarát, de időzítésben kiszámíthatatlan.

Nem determinisztikus viselkedés

A késleltetés drasztikusan megugorhat (100–300 ms) szemétgyűjtés vagy cache ürítés közben, ami kritikus telemetriai rendszerekben adatvesztést okozhat.

5 Probléma megfogalmazása

A fájlrendszerek a számítástechnikában különösen érzékeny területet jelentenek. Ha egy magasabb szintű program, például: illesztőprogram, alkalmazás vagy hálózati kapcsolat meghibásodik, a felhasználó általában csak időt vagy kényelmet veszít. Ezzel szemben egy olyan alacsony szintű rendszer, mint a fájlrendszer hibája adatvesztéshez, akár végleges károsodáshoz és anyagi kárhoz is vezethet. Ezért alapvető fontosságú az adatok biztonsága, integritásuk megőrzése és visszaállíthatósága.

A beágyazott környezetekben alkalmazott fájlrendszerekkel szemben rendkívül szigorú követelmények jelennek meg, amelyek számos szempontból eltérnek az általános célú rendszereknél megszokott elvárásoktól. Ezekben a rendszerekben a mérési adatok gyakran üzleti vagy kutatási szempontból értékesek, elvesztésük pedig közvetlen anyagi vagy kísérleti költséget jelent. Emiatt a fájlrendszer elsődleges célja nem a rugalmasság vagy a funkcionalitás bővíthetősége, hanem az adatok megbízható, ellenőrizhető és hosszú távon is konzisztens tárolása. Ez különösen igaz olyan extrém mérési környezetekben – például nagy gyorsulású kísérletek, magas légköri repülések, sugárzási hatásoknak kitett rendszerek – ahol a háttértár fizikai megbízhatósága is korlátozott.

A rendszernek mindenekelőtt kis nem illékony memóriafoglalással kell működnie, mivel a legtöbb mikrokontroller csupán néhány kilobájt RAM-mal rendelkezik, és a nagy metaadat-struktúrák fenntartása nem megengedett. A metaadat-használat minimalizálása nemcsak tárhelytakarékosági okból lényeges, hanem a determinisztikus működés érdekében is: a bonyolult indexek és könyvtárstruktúrák jelentősen növelnék az írási késleltetéseket és a fragmentáció kockázatát. Mindezek miatt egy mérésre optimalizált fájlrendszernek egyszerű, fix méretű és előre kiszámítható struktúrákat kell alkalmaznia.

Az adatok integritása szintén központi kérdés. A háttértárakat számos hiba érheti—bitfordulsa (bitflip) hibák, elhasználódott cellák, sugárzás okozta meghibásodások, valamint részlegesen befejezett írárok, amelyek áramkimaradás esetén különösen gyakoriak. Ezért a fájlrendszernek képesnek kell lennie minden tárolt rekord épségének ellenőrzésére, akár blokkszintű ellenőrzőösszegekkel vagy CRC-mechanizmussal. Fontos követelmény a gyors és egyértelmű hibadetektálás, valamint a redundancia akár több példány tárolásával, hogy sérülés esetén a visszaállítás determinisztikus módon elvégezhető legyen.

A determinisztikus működés elengedhetetlen minden valós idejű vagy közel valós idejű rendszerben. A fájlrendszernek garantálnia kell, hogy egy írási művelet meghatározott időn belül befejeződik, és nem használhat olyan háttérfolyamatokat, mint a szemétyűjtés (GC), a dinamikus tömörítés vagy a bonyolult visszajátszási algoritmusok. A modern flash-alapú tárolók – SD-kártyák, eMMC, NAND – belső vezérlői azonban gyakran nem determinisztikus viselkedést mutatnak. A elhasználódás elleni védelem, blokk töredezettség mentesítés (block remapping) és a memória ürítés (cache flush) olyan

késleltetéseket okozhatnak, amelyeket a fájlrendszer nem tud befolyásolni. Ezért mérési célú rendszerekben különösen fontos az olyan írási modell, amely a lehető legkevesebb véletlenszerű műveletet generálja, és minimalizálja a háttértár belső FTL rétegét terhelő tranzakciókat.

A rendszer megbízhatósága szempontjából kulcsfontosságú, hogy áramkimaradás vagy váratlan leállás esetén a fájlrendszer konzisztens maradjon. Elengedhetetlen, hogy a részleges írások ne okozzanak egész fájlrendszeren végiggyűrűző korrupciót, és a rendszer újraindítás után gyorsan, egyértelműen helyreállítható legyen. Ilyen környezetben a naplózás vagy a bonyolult visszajátszási protokollok használata gyakran túl drága, ezért a fájlrendszer struktúrájának eleve olyan módon kell robusztusnak lennie, hogy részleges írások mellett is értelmezhető maradjon.

Mindezek mellett az integrálhatóság és a karbantarthatóság is lényeges szempont. A fájlrendszernek úgy kell felépülnie, hogy könnyen portolható legyen különböző mikrovezérlőkre és háttértár-meghajtókra, minimális külső függőséggel. A fejlesztők számára átlátható, egyszerű és logikus struktúrát kell biztosítani, amely hosszú távon is könnyen diagnosztizálható és bővíthető.

A flash-alapú háttértárak sajátosságai tovább bonyolítják a helyzetet. A véletlenszerű írási műveletek jelentős mértékben csökkenthetik az élettartamot és megnövelik az írási amplifikációt, a korlátozott törlés-írás ciklusszám miatt pedig hosszú távú használat esetén könnyen kialakulhatnak elhasználódott blokkok. A fájlrendszer feladata ezért az, hogy minimalizálja a fragmentációt okozó írásokat, és olyan adatszerkezési modellt alkalmazzon, amely lehetőleg nagy, lineáris és előre kiszámítható írási mintát eredményez.

A fenti követelmények összessége világosan mutatja, hogy a hagyományos, általános célú fájlrendszerek nem alkalmasak ilyen környezetben. A fentebb tárgyalt fájlrendszerek pedig mind rendelkeznek olyan tulajdonságokkal vagy korlátozásokkal amik indokolják egy olyan speciális, mérési adatokhoz optimalizált fájlrendszer megalkotását, mint a ZINF, amely kifejezetten ezekre a kihívásokra ad választ.

6 ZINF funkcionális követelményei (Most Viable Product)

A ZINF adattároló rendszernek minden működési körülmény között konzisztens állapotot kell fenntartania, még váratlan energia-kiesés, rendszer újraindulás, instabil háttértár vagy részben befejezett írás esetén is. Az egyes blokkokhoz vagy rekordokhoz tartozó hibadektálási mechanizmusnak – különösen a CRC-alapú ellenőrzésnek – determinisztikusan és következetesen kell működnie. A redundáns példányok és a tükrözött adatok kezelése során biztosítani kell, hogy legalább egy hibátlan blokk megléte esetén az adott információ minden körülmények között visszanyerhető legyen. Fontos követelmény továbbá, hogy a rendszer megakadályozza a sérült, részben írt vagy inkonzisztens blokkok terjedését, és ne engedje, hogy ezek a teljes állapot helyreállíthatóságát veszélyeztessék.

A működés időbeli viselkedésének előre meghatározhatónak kell lennie, ezért minden adatírási műveletet olyan felső korláttal rendelkező késleltetéssel kell végrehajtani, amely legfeljebb egyetlen háttértár-blokk vagy szektor determinisztikus írási idejéhez igazodik. A rendszer nem alkalmazhat olyan háttérkarbantartási vagy újraszervezési lépéseket, mint például a szemétyűjtés, amelyek előre nem kalkulálható késleltetés-növekedést okoznának. Az olvasási műveleteknek ugyanilyen módon garantált és közel konstans idejű működést kell biztosítaniuk, beleértve a redundáns blokkok ellenőrzését és azok épségének validálását.

A megvalósításnak illeszkednie kell alacsony erőforrású mikrovezérlős környezetekhez, ezért a RAM-igényt minimálisra kell korlátozni: a működéshez szükséges állapotinformációk mennyisége nem haladhat meg néhány kilobájtot. A kódbasis méretének szintén néhány tíz kilobájt közötti tartományban kell maradnia, hogy egyszerűen illeszthető legyen tipikus MCU környezetekhez. A háttértár metaadatainak mérete csak a feltétlenül szükséges információkra korlátozódhat, hogy a teljes kapacitás döntő hányada tényleges mérési adatok számára maradjon elérhető. A tárolásnak szekvenciális vagy bővítés-alapú (append-only) mintát kell követnie, amely csökkenti a véletlenszerű blokkműveletek számát, és ezzel növeli a háttértár élettartamát.

A rendszer kialakításában alapvető követelmény a hordozhatóság és a modularitás. A működésnek jól definiált illesztőprogram-abstrakcióra kell épülnie, hogy a fájlrendszer mikrokontrollereken, PC-s szimulációs környezetben vagy virtuális háttértárakon egyaránt egységes módon használható legyen. A háttértárkezelő rétegnek egyszerűen cserélhetőnek kell lennie, akár SPI-alapú perifériáról, akár SSD-ről, akár HDD eszközről van szó.

A rendszernek úgy kell elkészülnie, hogy PC-n, emulált háttértáron vagy hibaszimulációs környezetben egyaránt tesztelhető legyen, az eredeti bináris struktúra megváltoztatása nélkül. A tesztelés során lehetőséget kell biztosítani különböző tipikus hibák – sérült szektorok, hibás ellenőrzőösszegek, hiányzó redundáns blokkok – szimulálására, amelyek révén a hibatűrés valós körülmények között is igazolható. A teljesítmény és a megbízhatóság összehasonlíthatósága érdekében biztosítani kell az automatikus tesztek

futtatását is.

Végül a rendszer karbantarthatósága érdekében a forráskódnak modulárisnak, átláthatónak és könnyen értelmezhetőnek kell lennie. A dokumentációnak tartalmaznia kell az adatblokkok felépítését, a működési állapotátmeneteket, valamint a helyreállítási eljárások részletes leírását. A rendszer belső logikájának kerülni kell a túlzott komplexitást, hogy hosszú távon is egyszerűen portolható, stabil és továbbfejleszthető maradjon.

7 ZINF nem funkcionális követelményei (Not-MVP)

A ZINF rendszer nem igényel komplex állománykezelési funkcionalitást, így nem feladata több különálló adatfolyam vagy blokk egyidejű támogatása. A háttér-karbantartási mechanizmusok szintén nem részei a követelményeknek: a rendszer nem végez tömörítést, terület-újraszervezést vagy egyéb háttérben futó optimalizációs folyamatokat, és nem elvárás a elhasználódás elleni védelem biztosítása sem. A fejlett indexelési és keresési képességek mellőzhetők, ezért a rendszer nem tartalmaz sem keresési algoritmusokat, sem időbélyeg-alapú indexelést, és nem feladata a rekordok gyors lokalizálása. Ugyanez igaz a metaadatokra is: nincs szükség összetett metaadat-struktúrákra vagy lekérdező rendszerekre.

A biztonsági funkciók sem képezik a minimális termék részét; a rendszer működéséhez nincs szükség titkosításra, hozzáférés-szabályozásra vagy jogosultságkezelésre, és nem követelmény hitelesítési, aláírási vagy kulcskezelési mechanizmusok beépítése sem. A konfigurálhatóság szintén korlátozott mértékben jelenik meg: nem szükséges több háttértár egyidejű kezelése vagy azok közötti dinamikus átkapcsolás. A platformfüggetlenség csupán minimális szinten elvárt, ezért nem igényel széles körű többplatformos támogatást, és elegendő egyetlen célplatformra optimalizált megvalósítás. Ennek megfelelően a rendszer nem követeli meg sem a POSIX-szerű interfészek használatát, sem bármilyen szabványos fájlrendszer API alkalmazását.

8 ZINF architektúrális felépítése

A ZINF felépítése több tükrözött tárolóterületből áll, amelyeken belül egy *metaadatblokk*, és több *adatblokk* helyezkedik el. A rendszer célja, hogy a mérési adatok folyamatos mentése során bármely hibatűró konfigurációban biztosítva legyen az adatok konzisztenciája és visszaolvashatósága, még áramkimaradás vagy hordozóhiba esetén is.

8.1 Logikai felépítés

A fájlrendszernek képesnek kell lennie CRC alapú hibadetektálásra és RAID-1 [17] alapú javításra. Tudnia kell rendszerüzeneteket tárolni. Nem írhat felül újraindítás után se adatokat, tudja pontosan, hogy hol maradt abba az írás.

8.1.1 Partíciók és tükrök

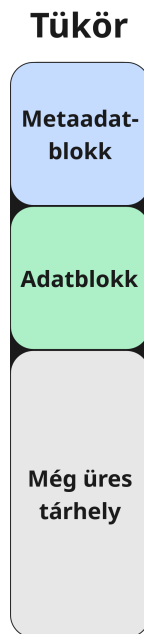
Partíció



8. ábra. Tükrök a partíción belül
Forrás: Saját kép

A ZINF egy fizikai partíciót használ, amelyen belül több egymástól független *tükör* (M darab, M-et a felhasználó adja meg, M eleme a valós pozitív egész páratlan számoknak és nagyobb mint 1) valósul meg. Minden tükör a teljes tárolóterület logikai másolata, vagyis a fájlrendszer minden lényeges szerkezeti eleme minden tükröben megtalálható. A redundancia mértékét a tükrök száma határozza meg, amit a háttértár tárhelyének észben tartásával kell meghatározni.

8.1.2 A tükrök belső struktúrája



9. ábra. Tükrök belső struktúrája
Forrás: Saját kép

Metaadatblokk A metaadatblokk a fájlrendszer globális metaadatait, illetve a rendszer üzeneteket tartalmazza. Itt helyezkedik el az utolsó írt adatblokk mutatója, az utolsó írt rendszer üzenet mutatója, illetve maguk a rendszer üzenetek is.

Egy szektor rendszerint 512 bájtos, így az utolsó írt szektor mutatójának tárolására kezdetben elegendő volt 4 bájt (unsigned 32-bit integer). Ennek legnagyobb értéke 4 294 967 295, amelyet 512-vel (egy szektor átlagos mérete bájtban mérve) felszorozva a fájlrendszer gyakorlati maximum mérete 2 TB (tükrözés nélkül). Tükrözéssel együtt ez legalább 6 TB fizikai tárhelyet igényelne.

Bár ez rengeteg mérési adat tárolására alkalmas lenne, célszerű ezt a jövőben skálázni. Ha a címezést 64 bitre (8 bájt) növeljük, 8 388 608 PB adat tárolható (tükrözés nélkül). Jelenleg elegendő a 32 bites címezés.

A metaadat szektorok száma ebben az iterációban mindig 2, ami erősen korlátozza az eltárolható rendszerüzenetek számát. Két szektor összesen 1024 bájt, amelyből:

- 4 bájt az utolsó szektor mutatója,
- 2 bájt az utolsó üzenet indexe (max. 65 535),
- 4 bájt a CRC-32,

így összesen 1014 bájt marad üzenetek számára, amely 1 bájtos üzenetkódok esetén 1014 üzenetet jelent.

Egy kisebb rendszer számára ez elegendő, nagyobb rendszereknél azonban bővítés szükséges. Az utolsó üzenet indexének módosítása nélkül 127 szektor használható fel erre a célra, ami 64 516 bájtnyi, 63 kB hasznos kapacitás. A jövőben ezt a felhasználó számára módosíthatóvá kell tenni, hogy annyi üzenetet tárolhasson el, amennyit akar.

**Metaadat-
blokk**



10. ábra. Metaadatblokk belső struktúrája
Forrás: Saját kép

Adatblokk Az, hogy az adott blokkokban milyen típusú adatok kerülnek tárolásra, nem célszerű maguk az adatok mellett tárolni. Az adatok azonosításához ezért szükséges a típus tárolása a fejlécben. A típus 1 B, ami elegendő 255 különböző mérési struktúra megkülönböztetésére. A tárolóban a típus és a struktúra kapcsolata nem jelenik meg, csak a mikrovezérlő programjába fordított konfigurációban (config.c, config.h). Ugyanez a konfiguráció megtalálható a számítógépen is, így tudjuk a tárolt adatokat visszaállítani.

Kezdetben a konfiguráció egy egyszerű header–source fájl páros, később ezt egy YAML (A YAML egy könnyen olvasható és írható adatsorosítási formátum. A rendszerfejlesztés és -menedzsment világában széles körben elfogadott szabvánnyá vált. A szintaxisa intuitív és annak képessége szerkezeti információk egyértelműen népszerűvé tették a szoftverkonfigurációban, különösen a DevOps környezetekben és programozásban) [18] alapú konfiguráció fogja majd felváltani, amelyből automatikusan generálódik a program számára a header és a source fájl. A YAML fájlra azért van szükség, mert így a programozáshoz kevésbé értő felhasználók is könnyen tudják használni a ZINF-et. YAML fájlokban beszédes, könnyű konfigurációt lehet írni és a JSON fájlokkal szemben lehet bennük kommenteket is írni. Ez a konstrukció lehetővé teszi az tárolt adat méretének csökkentését, ugyanakkor komplex adatstruktúrák mentését is.

A ZINF egyik fontos tulajdonsága, hogy a mentés szekvenciális sorrendben történik. A már elmentett adatokat nem módosítjuk, és egy kezdetben megnyitott blokkot utólag bővíteni sem lehet; új blokkot kell nyitni akkor is, ha az előző nem telt meg. Ez az írás mód a Copy-on-Write elvet követi, ami magas szintű biztonságot tesz lehetővé, de ezt csak az adatblokkokra használjuk. A metaadatblokkok mindig felülírásra kerülnek. Ugyan

a blokkok a tükrökön belül közvetlenül egymás után helyezkednek el. Ennek köszönhetően pontosan ismerjük az eredmények időbeli sorrendjét. Így egy fix mintavételezésű mérési sorozatnál elegendő az első írásnál rögzíteni az időbélyeget. Nagyobb precizitás igénye esetén természetesen időközönként szinkronizációs időpontokat is rögzíthetünk.

A blokk alatti réteg, a legkisebb elemi réteg a szektor. Egy blokk több szektorból is állhat, de tipikusan 3 feladatkör van amit be kell tölteni: a nyitás, a tárolás és a zárás. Ezeket a funkciókat egy szektor is betudja tölteni.

1. **Alfaszektor** Ez a szektor minden adatblokk elején található, ez a nyitószektor:
 - *fejléc* – a blokk típusát, időt tartalmazó mezők,
 - *adat* – mérési adatok
2. **Köztes adatszektorok** Ezek csak mérési adatokat hordoznak
3. **Omega szektor** Az Omega szektor minden adatblokk lezáró eleme. Két fő részből áll:
 - *adat* – a blokk utolsó tartalmi elemei,
 - **CRC** – az egész blokk integritását biztosító CRC-32 ellenőrzőérték.

Ez a CRC határozza meg, hogy az adott tükrökben lévő adott blokk ép-e vagy sérült.

Adatblokk



11. ábra. Adatblokk belső struktúrája
Forrás: Saját kép

Elvesztett tárhely Ennek a konstrukciónak az a hátránya, hogy ha egy adatblokk nem telik meg teljesen (mert a rekordok mérete nem osztója a sektorméretnek), a fennmaradó bájtok elvesznek. A maximálisan elveszthető bájtok száma egy mentés esetén:

$$\text{Veszteség}_{\max} = \text{adatstruktúra}_{\text{méret}} - 1 \quad [\text{bájt}]$$

Példaként vegyünk egy tipikus mérési struktúrát, ahol a rekordonként eltárolandó adatmennyiség 14 bájt:

```
typedef struct {
    float temperature, humidity;    // 2 * 4 bájt
    int16_t a_x, a_y, a_z;          // 3 * 2 bájt
} dataset_t;                       // Összesen: 14 bájt
```

1. Szektor szintű elemzés

Vizsgálva a hatékonyságot egyetlen, 512 bájtos szektor esetén.

Bemenő paraméterek:

- Rekord mérete: 14 bájt
- Szektor mérete: 512 bájt

Egy szektorban biztonságosan tárolható teljes rekordok száma:

$$N = \left\lfloor \frac{512}{14} \right\rfloor = 36 \text{ db}$$

A hasznos adatmennyiség szektoronként:

$$36 \times 14 = 504 \text{ bájt}$$

Elveszett terület szektoronként:

$$512 - 504 = 8 \text{ bájt}$$

2. Rendszer szintű hatékonyság

Kiszámolva, mennyi hasznos adatot tudunk tárolni, mire összesen **1 MiB** tárhelyet veszünk el.

Szükséges szektorok száma 1 MiB (1 048 576 bájt) veszteséghez:

$$\frac{1\,048\,576}{8} = 131\,072 \text{ szektor}$$

Ez alatt eltárolt **hasznos** adatmennyiség:

$$131\,072 \times 504 \text{ B} \approx \mathbf{63 \text{ MiB}}$$

Hatékonysági mutató: Minden 1 MiB veszteségre 63 MiB hasznos adat jut. A tároló kihasználtsága:

$$\eta = \frac{63}{63 + 1} \approx \mathbf{98,44\%}$$

Ez az 1,56%-os kapacitásvesztés az ára a biztonságos, blokk-alapú, determinisztikus írásnak. Az arány természetesen függ a választott adatstruktúra méretétől.

8.2 Működési felépítés

8.2.1 Rétegek

Rétegeit tekintve a ZINF két nagy rétegből áll.

1. **Tároló réteg (Storage Layer):** Ez a réteg a legmagasabb absztrakciós réteg. Az alatta lévő illesztő (driver) rétegre épít, amivel egy szabványos interfészen keresztül kommunikál. Itt van megvalósítva minden üzleti logika.
2. **Illesztő réteg (Driver Layer):** Az illesztő réteg az opaque tervezési mintát (opaque design pattern) követi, tehát a driver.h-ből exportált függvények alatt nem számít milyen logika van. Ennek köszönhető az, hogy ezt a réteget tudja használni a felette lévő, attól függetlenül, hogy valójában milyen fizikai eszközön kell adatot tárolni. Az illesztő réteg alatt már csak a fizikai eszköz található.

8.2.2 Programozási nyelv

Programozási nyelvnek kettő merült fel: a Rust és a C. Mind a kettő rendkívül alkalmas ennek a feladatnak az elvégzésére mivel: nagyon alacsony hardveres hozzáférést tesznek lehetővé, rendkívül gyorsak és kevés tárhelyet vesznek igénybe fordítás után. Ami miatt a C mellett döntöttem az igazából két ok. Az első, hogy eddig ezt használtam többet, sokkal rutinosabban mozogtam a szintaxisában és gondolkodás módjában mint a Rust-éban. A második ok az a támogatás. Mivel a C már egy rendkívül régóta jelen lévő nyelv nagy a közössége és óriási tudást halmoztak fel róla az interneten, míg a Rust, ugyan egy erősödő felhasználóbázissal bír, de még nincs mindenről elég magyarázat és információ az interneten.



12. ábra. C nyelv logója

Forrás: <https://www.britefish.net/c-programming-language/>

8.2.3 Fordítási környezet

A projekt fordítási környezetének kiválasztásakor két lehetőség merült fel: a hagyományos Makefile alapú rendszer és a modernebb, magasabb szintű CMake. Mindkettő alkalmas C projektek fordítására, viszont eltérő filozófiát és absztrakciós szinteket képviselnek.

A Makefile egy rendkívül egyszerű, közvetlen és alacsony szintű eszköz. A szabályok, célok, függőségek és parancsok teljes egészében a fejlesztő kezében vannak. Ennek köszönhetően a fordítási folyamat pontosan és determinisztikusan kézben tartható, ami különösen fontos olyan beágyazott rendszereknél, ahol a build folyamat gyakran speciális

eszközláncot (`arm-none-eabi-gcc`, `openocd`, linker script), manuális memóriacímzést és hardverközeli beállításokat igényel. A `Makefile` további előnye, hogy minimális környezeti függőséggel rendelkezik: bármilyen Unix alapú rendszerben azonnal futtatható, nem igényel külön konfigurációs lépéseket, generátorokat vagy külső modulokat.

Ezzel szemben a `CMake` egy magasabb szintű, platformfüggetlen build-rendszer, amely a forrásból különböző build-leírásokat (`Makefile`, `Ninja`, Visual Studio projektfájlok stb.) generál. Kifejezetten nagy és összetett projektek kezelésére kiváló, azonban beágyazott környezetben sokszor túlzott absztrakciót vezet be. A speciális fordítókapsolók, linker scriptek és eszközláncok kezelése több konfigurációs lépést igényel, és a generált `Makefile`-ok kevésbé átláthatók, mint a kézzel írottak. További hátrány, hogy a `CMake` általában külön build könyvtárat, konfigurációs fázist és extra rétegeket vezet be, ami lassítja és bonyolítja a fejlesztési folyamatot egy kis méretű firmware esetében.

Azért döntöttem a `MakeFile` mellett, mert már dolgoztam vele, a `CMake`-el nem, illetve kényelmesnek bizonyult a használata számomra, főleg Linuxon való fejlesztés során. Beágyazott rendszerekre (`Mimike-II Rev-I`) való fejlesztés közben még az `arm-none-eabi-gcc`-t a későbbi fejlesztés során már a hagyományos GNU Compiler Collection-t (`gcc`) használtam fordítónak.



13. ábra. GNU logója

Forrás: <https://www.britefish.net/c-programming-language/>

8.3 A felhasználói interfész

A rendszer széleskörű és hatékony alkalmazhatóságának egyik legfontosabb feltétele a könnyű konfigurálhatóság. Ennek biztosítására elengedhetetlen egy olyan platformfüggetlen kliensalkalmazás fejlesztése, amely lehetővé teszi a ZINF rendszer paraméterezését mélyebb programozási előismeretek nélkül is.

A felhasználói élmény optimalizálása és a szakértői igények kielégítése érdekében az alkalmazás kettős nézetet (`dual-view`) biztosít:

1. **Grafikus felület (GUI):** Egy intuitív, vizuális beállítófelület az általános felhasználók számára.
2. **YAML szerkesztő:** Egy beépített, szintaxiskiemeléssel támogatott nézet a komplexebb, finomhangolást igénylő konfigurációkhoz.

8.3.1 A választott technológiai stack: Tauri és Svelte

A fejlesztéshez a **Tauri** háttérrendszert (backend) és a **Svelte** felhasználói felület (User Interface, UI; frontend) keretrendszereket választottam. Ez a kombináció lehetővé teszi modern, webes technológiákra épülő, mégis natív teljesítményt nyújtó asztali alkalmazások létrehozását. A rendszer architektúrája két fő rétegre bontható:

- **Háttérrendszer (Rust):** Az alacsony szintű szolgáltatásokat, az operációs rendszerrel való kommunikációt és a fájlrendszer-műveleteket egy Rust nyelven írt, nagy teljesítményű és memóriabiztonságot garantáló réteg biztosítja.
- **Felhasználói felület (Svelte):** A vizuális megjelenésért és a felhasználói interakciókért a webfejlesztésben is elterjedt Svelte keretrendszer felel.

8.3.2 A technológiai döntés indoklása

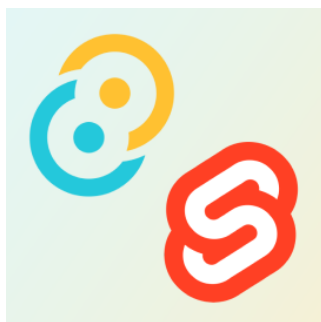
Tauri vs. Electron A legelterjedtebb alternatívával, az Electronnal szemben a Tauri jelentős előnyöket kínál erőforrás-gazdálkodás terén. Míg az Electron minden egyes alkalmazáshoz mellékel egy teljes Chromium motort, addig a Tauri az operációs rendszer saját, beépített megjelenítőjét (WebView) használja. Ez a megközelítés drasztikusan csökkenti a fordított alkalmazás méretét és a futáskori memóriaigényt.

1. táblázat. A Tauri és az Electron keretrendszerek összehasonlítása

Szempont	Tauri (Választott)	Electron
Renderelő motor	OS natív WebView (pl. Kit2GTK)	Web- Beépített Chromium
Backend nyelv	Rust	Node.js
Bináris méret	Kicsi (< 5 MB)	Nagy (> 50 MB)
Memóriaigény	Alacsony	Magas
Biztonság	Magas	Átlagos

Svelte vs. React/Vue A felhasználói felület megvalósításához a Svelte keretrendszert választottam a React-tel vagy a Vue-val szemben. A Svelte legfőbb előnye, hogy a munkát fordítási időben (compile-time) végzi el, nem pedig futásidőben egy virtuális dokumentumobjektum-modell (Document Object Model, DOM) (Virtual DOM) segítségével. Illetve szintaktikailag és működésileg is egy sokkal modernebb rendszer a React/Vue-val szemben.

- **Fenntarthatóság:** A kód tisztább, kevesebb sablon kódot (boilerplate) igényel.
- **Teljesítmény:** A virtuális DOM hiánya miatt a DOM-manipulációk gyorsabbak, ami a vizuális konfiguráció és a YAML nézet közötti valós idejű szinkronizációt zökkenőmentesebbé teszi.



14. ábra. Tauri (bal) és Svelte (jobb) logója

Forrás: <https://sveltethemes.dev/auros-one/tauri-sveltekit>

9 Fejlesztés menete / Történeti bemutatás

9.1 Interakció blokk alapú eszközökkel

A fejlesztés ott vette kezdetét, hogy a mikroprocesszorra (ATSAMD21G18AU) kötött mikro SD-kártyával kellett felvenni a kapcsolatot. Az SD-kártyák esetén a tárhely méretétől függően eltérő címezési módszert kell alkalmazni. Amennyiben a kapacitás kisebb vagy egyenlő 2 GB-tal, akkor SD kategóriába tartozik [19], és a szektor sorszámaival lehet címezni. Ha ennél nagyobb méretű, akkor SDHC/SDXC kategóriába esik, ahol a szektor kezdő bájtjának sorszámaát kell megadni. Ez a címezés később az illesztőprogram rétegbe kerül, a fejlesztőknek nem kell vele foglalkozni, ha nem akarnak saját illesztőprogramot implementálni.

Függetlenül attól, hogy milyen háttértár található a fájlrendszer alatt, a későbbiekben (például RAID-elés során) szükség lehet arra az információra, hogy mekkora teljes tárhely áll rendelkezésre. Az első implementáció idején a kódba közvetlenül bele volt égetve az SD protokoll, amely SPI-re épült, így a rendszer kezdetben kizárólag SD-kártyára tudott írni.

9.2 Blokkok és szektorok

Miután már működött az SD-kártyára történő írás, elkezdődhetett a tényleges rendszer implementációja. Az elsőként készült modul a metaadat szektor volt, amely tárolta az utolsó írt szektor sorszámaát, illetve a rendszer logjait. Itt ez még csak egy szektor volt, később bővült kettőre. Illetve ezután egyből létre tudtam hozni az első adatszekort. Akárcsak a metaadatszekornál itt is csak egy szektor volt elsőre elérhető, tehát a RAM-ból az SD-kártyára írható adat maximum 512 bájt lehetett. Ezt hamar bővíteni kellett, mivel annak ellenére, hogy a rendszer tervezéstől fogva nem teljesen dinamikus működésű, de ez nagyon lekorlátozta a működését, mivel külön mérési eredményeknek külön-külön mérési struktúráknak más-más mennyiségű adatot kell egyszerre mentenie. Azután hogy bővítettem az adatszekort blokkra egyből bővítettem a metaadatszekort is blokkra. A rendszer üzeneteknek fenntartott helyet még akkor sem sikerült megtölteni, mikor a Mimike-II Rev-I-en futott a ZINF.

9.3 Visszaolvasás

Miután a mentés már működött, implementálni kellett az adatok visszaolvasását is. Mivel csak SD-kártyáról való olvasásra volt szükség, az első megoldás egy Bash script volt, amely hexdump-ot készített és CSV fájlokat generált a tartalmából. Két CSV készült: egy az adatoknak és egy a metaadatoknak. A Bash scriptet csak PC-n használjuk, mivel ilyen szintű olvasásra nincs szükség mérés közben. Az az olvasás ami mérés során is történik

C-ben van megvalósítva.

Ez a megoldás azonban gyorsan alkalmatlannak bizonyult: nem volt hordozható, nem volt elég rugalmas, és a CRC-32 bevezetésekor végleg lecserélésre szorult.

9.4 Kód absztrakció

Ahogy a rendszer egyre bővült, felmerült a fenntarthatóság és bővíthetőség problémája. Szükségessé vált a különböző rétegek szétválasztása. Két fő réteg alakult ki:

- a driver réteg – az SD-kártyával való kommunikációhoz,
- a core réteg – a ZINF logikájával és működésével.

Egy szabványosított driver interfész megteremtette annak lehetőségét, hogy a kód teljesen modulárisra váljon.

9.5 Validálás

Bár már lehetett mérni, tárolni és visszaolvasni adatokat, nem volt biztosítva azok helyességének ellenőrzése. A legegyszerűbb és leggyorsabb megoldás erre a CRC-32 (cyclic redundancy check) bevezetése volt. Ezt széles körben alkalmazzák hálózati kommunikációban (pl. Ethernet), valamint fájlrendszerekben és archív formátumokban (pl. FDDI, ZIP).

Innentől minden adatblokk végén megtalálható a CRC-32, és minden olvasáskor ellenőrzésre kerül. Mivel a Bash scripttel ez már körülményes volt, a validálást C-ben kellett megvalósítani.

9.6 Visszaállítás / javítás

A CRC-32 segítségével megállapítható, hogy egy blokk hibás-e, de nem biztosít lehetőséget a javításra. Ellentétben például a hálózati kommunikációval az adatot itt nem tudjuk újra elkérni. Ezt a problémát lehet áthidalni a RAID-1-el. Segítségével nagy mértékben lehet növelni az adatok biztonságát, bár nehéz döntés volt bevezetni, hiszen eddig minden bájton spórolni próbáltam. A redundáns tárolás azonban jelentősen növeli a visszaállíthatóság esélyét, ezért az „elvesztett” tárhely ellenére is megérte az alkalmazása.

Az első implementációban a tükrök közvetlenül egymás után helyezkedtek el a kártyán, és a metaadat szektor nem volt tükrözve, mivel a gyakori olvasások miatt felesleges sokszorozásnak tűnt.

9.7 Első alkalmazás valós környezetben

A ZINF ekkori állapotában már tartalmazta az SD-re való írást, a metaadat szektort, a CRC-32-t és a RAID 1-et. Ez a verzió került használatra a Juhász Jenő Szakkollégium Mimike-II Rev-I CanSat projektjében.

A rendszer UV-intenzitást, 9 tengelyes gyorsulásmérőből származó adatokat, hőmérsékletet, valamint natív GNSS-ből szűrt adatokat tárolt és olvasott vissza. Éles bevétele a Rev-I-nek sosem volt, de a tesztek során a működés stabilnak bizonyult, egyetlen bájt sem sérült vagy veszett el. A leghosszabb teszt hozzávetőleg 10 óra volt, ami alatt a fentebb említett méréseket másodpercenként végezte el, majd percenként mentett az SD-kártyára.

9.8 Illesztőprogram interfész

Eddig a ZINF kizárólag a Mimike-II projekt részeként létezett. Innentől önálló GitHub-projektte vált, és ki vettem alóla az SD-kártya-specifikus implementációt, mert elkészült a szabványosított driver interfész, majd az első új driver is: a Linux loopback eszközhöz. Ezek virtuális meghajtók, amelyek pontosan paraméterezhetők, így ideálisak fejlesztési környezetben.

9.9 Konfigurálhatóság, felhasználóbarátság

A korábbi verziókban számos paraméter közvetlenül a kódba volt égetve. Ezeket ki kellett szervezni a config.c/h fájlokba, hogy a rendszer könnyebben konfigurálható legyen, és előkészítse a YAML alapú konfiguráció jövőbeli bevezetését. Mostantól a ZINF író és olvasó oldala is közös konfigurációból működik.

9.10 Random írásvédelem

A RAID 1 eddigi implementációja közvetlen egymás utáni tükrözést alkalmazott blokkonként. Ha azonban valamilyen véletlenszerű írás történne – hardveres vagy szoftveres hiba miatt –, a már elmentett adat felülíródhatna.

A tárhely és a felhasználó által megadott tükrök számának hányadosából kiszámolható, hogy mekkora távolságokra kell elhelyezni a tükröket. A tükrök száma egy 1-nél nagyobb, minél nagyobb páratlan egész szám kell legyen, hogy többségi szavazással eldönthessük, melyik példány a helyes. A végső darabszámot a tárhely, a misszió alatti várt adatmennyiség és a hibavalószínűség alapján érdemes meghatározni. Alapvetően a tárhely per várható adatmennyiség adja ki az ajánlott számot. Ebbe persze beleszól a tükrözés extra ideje is.

9.11 Utolsó szektormutató réteges védelme

A következő probléma az volt, hogy az utolsó szektormutató értéke nem redundáns: csak egyetlen helyen, a metaadat szektor elején tárolódik. Bár CRC-32 segít megmutatni, hogy hibás-e, de nem teszi lehetővé a javítást. Erre több megoldás létezik:

- végig lehet iterálni az összes szektort az utolsó érvényes megtalálásához,
- vagy több példányban is lehet tárolni az értéket.

Utóbbi esetben verziószámozásra is szükség lehet, hogy áramkimaradás esetén eldönthessük, melyik a helyes érték. Ez azonban extra tárhelyet igényel. Ez a megoldás lett kiválasztva, egyelőre a verziószám nélkül.

10 A fájlrendszer tesztelése és validációja

A ZINF fejlesztése során a legfontosabb szempont az adatintegritás és a determinisztikus működés biztosítása volt. Így a hagyományos fájlrendszerekkel szemben az írási sebesség növelése nincs a funkcionális követelmények között. A rendszer helyességének igazolására egy többszintű tesztelési stratégiát dolgoztam ki.

10.1 Tesztelési módszertan és környezet

A validációs folyamat a negatív tesztelés (negative testing) elvét követi: nemcsak azt vizsgálom, hogy a rendszer képes-e helyesen működni ideális körülmények között, hanem elsősorban azt, hogyan reagál a szándékosan előidézett hibákra, mint például a részleges írás vagy a szektorhiba.

A tesztkörnyezet a fejlesztés során megvalósított Linux loopback eszközre épül. Ez lehetővé teszi, hogy a fájlrendszer egy virtuális lemezképfájlon (image file) dolgozzon, ahol a hardveres hibák szoftveresen, kontrollált módon reprodukálhatók. A tesztkörnyezet elemei:

- **DUT (Device Under Test, tesztelt eszköz):** A ZINF fájlrendszer platformfüggetlen magja (core), amely determinisztikus működést garantál.
- **Illesztő:** Linux `/dev/loopX` eszközre író absztrakciós réteg, amely a hardvereszközt helyettesíti.
- **Validáló (Verifier):** Egy külső, a fájlrendszertől független ellenőrző szoftver, amely bit szinten validálja a lemez tartalmát és a CRC értékeket.

10.2 Teljesítménymérés és determinisztikusság

A mérési adatgyűjtő rendszerekben kritikus a maximális írási késleltetés ismerete és a determinisztikus időzítési viselkedés garantálása. A teljesítmény-mérő tesztek (benchmark) célja a rendszer áteresztőképességének és késleltetésének mérése volt különböző terhelési viszonyok mellett. A mérések során a rendszer viselkedését az alábbi szempontok szerint vizsgáltam:

- **Áteresztőképesség:** A mérések igazolták, hogy a ZINF képes kezelni a folyamatos adatfolyamot, miközben a *Copy-on-Write* (CoW) mechanizmus biztosítja az adatok biztonságát.
- **Determinisztikusság:** A tesztek megerősítették, hogy a ZINF nem alkalmaz olyan háttérfolyamatokat (pl. Garbage Collection), amelyek váratlan késleltetési tüskéket okoznának, így a működése előre kiszámítható.

-
- **Hosszú távú stabilitás:** A tesztek során több millió írási ciklust szimuláltam, biztosítva, hogy a memóriahasználat statikus maradjon, és ne lépjen fel fragmentációból adódó lassulás.

10.3 Hibatűrés és Integritás vizsgálata

Ez a tesztfázis a rendszer legkritikusabb elemét, a RAID-1 helyreállító képességét és a CRC-32 ellenőrzést vizsgálja. A fejlesztett hibainjektáló szoftver segítségével szándékos hibákat idéztem elő a virtuális tárolón, szimulálva a valós úripari környezeti hatásokat és az instabil háttértárak viselkedését.

10.3.1 Hibainjektálási szenáriók

A tesztprogram véletlenszerűen kiválasztott szektorokon hajtott végre roncsolási műveleteket, ellenőrizve a ZINF reakcióját:

1. **Bitfordulás (Bitflip):** A szektor tartalmának véletlenszerű módosítása, amely a sugárzás okozta hibákat vagy a flash memória celláinak degradációját szimulálja.
2. **Megszakadt írás (Torn Write):** A szektornak csak egy része kerül kiírásra. Ez a teszt ellenőrzi, hogy áramkimaradás esetén a rendszer felismeri-e a félbehagyott tranzakciót, és megakadályozza-e a sérült blokkok terjedését.
3. **Tükör inkonzisztencia:** A RAID-1 tükrök eltérő adatot tartalmaznak. A rendszernek képesnek kell lennie a hibás példány azonosítására a CRC ellenőrzőösszeg alapján, és a helyes példányból való visszaállításra.

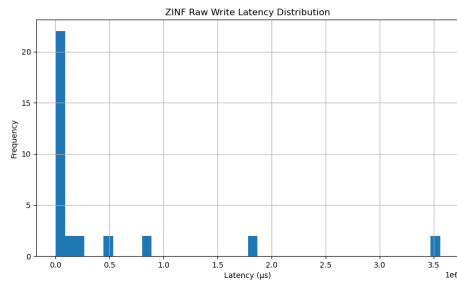
A tesztek során fokozatosan növeltem a károsított szektorok számát (300, 1300, 2800), ezzel keresve azt a határt aminél a visszaállítás már nem volt lehetséges. Ez abban a szituációban fordulhat elő, ha mind a 3 szektor meghibásodott. Minden mérést 10-szer végeztem el majd ezeket az eredményeket átlagoltam.

10.3.2 Helyreállítás és Validáció

A hibák injektálása után a visszaolvasó (validáló) szoftverrel ellenőriztem a fájlrendszer konzisztenciáját. A validáció során a szoftver minden blokkot beolvasott, és összevetette a számított CRC-32 értéket a tárolt értékkel. A tesztek igazolták, hogy:

- A rendszer minden esetben detektálta a sérült szektorokat a CRC mechanizmus segítségével.
- Amennyiben legalább egy tükör sértetlen maradt, a ZINF sikeresen visszaállította az adatokat, biztosítva a folyamatos működést.

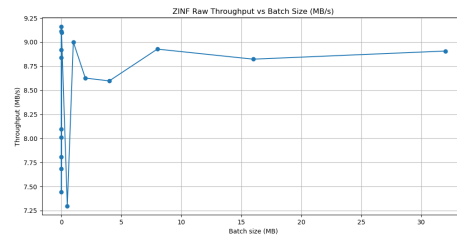
- A részlegesen írt blokkokat a rendszer érvénytelennek minősítette, így megőrizte a fájlrendszer globális konzisztenciáját.



15. ábra. Késleltetés hisztogram

Késleltetés

Ahogy a 15-es ábra is mutatja, a késleltetés aránylag konzisztens. A legmagasabb érték is csupán 3,5 másodperc volt, de ennek előfordulása rendkívül ritka. Az átlagos késleltetés 100 µs körül alakul.



16. ábra. Áteresztőképesség

Áteresztőképesség

Az értékek 7,25 és 9,25 MB/s között mozognak (16. ábra). A kezdeti ingadozást a Linux I/O optimalizációja okozza kis adatmennyiségeknél. Nagyobb adatmennyiségnél a sebesség stabilizálódik (8,5–9 MB/s), ami a feladat szempontjából megfelelő.

Hiba helyreállítási tesztek A Linux loopback eszközt ezekhez a tesztekhez 2 MB tárhellyel és 3 tükörrel használtam (4096 felhasználható, 1365 logikai szektor).

```
(zalanvdsz@aranykaca src)$ sudo ./reader /dev/loop0

== ZINF Reader ==
Input file   : /dev/loop0
Sector size  : 512 bytes
Total sectors: 16384
RAID mirrors : 3
RAID offset  : 5461

== Supersector ==
Last sector  : 4097
Last msg idx  : 0
Ring full    : 0

== Scanning Log Sectors ==

Logical sector 2
-----
Mirror 0 @ physical 2      Header: 0x01 Stored CRC: 0xC55B5092 Calc CRC: 0xC55B5092 [ok]
Mirror 1 @ physical 5463   Header: 0x01 Stored CRC: 0xC55B5092 Calc CRC: 0xC55B5092 [ok]
Mirror 2 @ physical 10924  Header: 0x01 Stored CRC: 0xC55B5092 Calc CRC: 0xC55B5092 [ok]
-> Result: VALID (mirror 0)

Logical sector 3
-----
Mirror 0 @ physical 3      Header: 0x01 Stored CRC: 0xC55B5092 Calc CRC: 0xC55B5092 [ok]
Mirror 1 @ physical 5464   Header: 0x01 Stored CRC: 0xC55B5092 Calc CRC: 0xC55B5092 [ok]
Mirror 2 @ physical 10925  Header: 0x01 Stored CRC: 0xC55B5092 Calc CRC: 0xC55B5092 [ok]
-> Result: VALID (mirror 0)

Logical sector 4
-----
Mirror 0 @ physical 4      Header: 0x01 Stored CRC: 0xC55B5092 Calc CRC: 0xC55B5092 [ok]
Mirror 1 @ physical 5465   Header: 0x01 Stored CRC: 0xC55B5092 Calc CRC: 0xC55B5092 [ok]
Mirror 2 @ physical 10926  Header: 0x01 Stored CRC: 0xC55B5092 Calc CRC: 0xC55B5092 [ok]
-> Result: VALID (mirror 0)

Logical sector 5
-----
Mirror 0 @ physical 5      Header: 0x01 Stored CRC: 0xC55B5092 Calc CRC: 0xC55B5092 [ok]
Mirror 1 @ physical 5466   Header: 0x01 Stored CRC: 0xC55B5092 Calc CRC: 0xC55B5092 [ok]
Mirror 2 @ physical 10927  Header: 0x01 Stored CRC: 0xC55B5092 Calc CRC: 0xC55B5092 [ok]
-> Result: VALID (mirror 0)

Logical sector 6
-----
Mirror 0 @ physical 6      Header: 0x01 Stored CRC: 0xC55B5092 Calc CRC: 0xC55B5092 [ok]
Mirror 1 @ physical 5467   Header: 0x01 Stored CRC: 0xC55B5092 Calc CRC: 0xC55B5092 [ok]
Mirror 2 @ physical 10928  Header: 0x01 Stored CRC: 0xC55B5092 Calc CRC: 0xC55B5092 [ok]
-> Result: VALID (mirror 0)
```

17. ábra. Olvasás visszajelzése

Forrás: Saját kép

```

=== Summary ===
Valid sectors      : 4096
Corrupted sectors  : 0
Unrecoverable (all mirrors bad): 0
Mirrors used       : 3
Output CSV         : ./out/payload.csv, ./out/meta.csv

```

18. ábra. Hibamentes olvasás

Normál működés

Hiba nélküli olvasás esetén a rendszer visszaadja a kiexportált CSV adatokat, hibaiüzenet nélkül.

```

=== Summary ===
Valid sectors      : 4096
Corrupted sectors  : 0
Unrecoverable (all mirrors bad): 0
Mirrors used       : 3
Output CSV         : ./out/payload.csv, ./out/meta.csv

```

19. ábra. Alacsony hibainjektálás

Alacsony hibaarány

300 sérült szektor esetén a rendszer képes volt az összes hibát teljesen korrigálni a tükrök segítségével. Adatvesztés nem történt.

```

=== Summary ===
Valid sectors      : 4095
Corrupted sectors  : 1
Unrecoverable (all mirrors bad): 1
Mirrors used       : 3
Output CSV         : ./out/payload.csv, ./out/meta.csv

```

20. ábra. Közepes hibainjektálás

Közepes hibaarány

1300 sérült szektornál előfordult, hogy mindhárom tükör sérült egy adott pozícióban. A többségi szavazás itt már nem működött mindenhol. Ez kb. 0,0732%-os adatvesztést jelentett.

```

=== Summary ===
Valid sectors      : 4079
Corrupted sectors  : 17
Unrecoverable (all mirrors bad): 17
Mirrors used       : 3
Output CSV         : ./out/payload.csv, ./out/meta.csv

```

21. ábra. Magas hibainjektálás

Magas hibaarány

Extrém terhelésnél (minden 1,5. szektor sérült), 17 logikai szektor vált olvashatatlaná. Ez 1,245%-os adatvesztés, ami a hiba mértékéhez képest kiváló eredmény.

10.4 Valódi környezetben bekövetkező hibák

Valós felhasználási környezetben, az űrben a leggyakrabban előforduló esemény az egyszer bekövetkező hiba (SEU, Single Event Upset). Ekkor egy darab ionizált részecske ütközik a tárhellyel, ami az ütközés környezetében megváltoztatja a töltéseket, bitfordulás (bitflip) jön létre. Ezek az ionizált részecskék legtöbbször a galaktikus kozmikus háttérsugárzásból (Galactical Cosmic Radiation), Napkitörésből (Solar Particle Events), Föld mágneses öveiből (Van Allen övek) származnak. Valós környezetben az előfordulása egy teljes szektor károsodásának megközelítőleg zérus. Az általam végzett tesztek ehhez képest sokkal nagyobb károsodásokat okoztak. Így azt a következtetést lehet levonni, hogy a ZINF jelenlegi formájában is nagyfokú megbízhatóságot tanúsít.

11 Összefoglaló

A dolgozatban bemutatott ZINF fájlrendszer egy olyan speciális, beágyazott mérésadat-gyűjtő környezet igényeire adott válasz, ahol az energiaellátás, a rendelkezésre álló erőforrások és a háttértárak megbízhatósága erősen korlátozott. A probléma onnan indult, hogy a CanSat- és CubeSat-jellegű rendszerekben a hagyományos fájlrendszerek (például FAT32, LittleFS, SPIFFS) nem képesek garantálni a determinisztikus működést, a nagyfokú hibatűrést és a minimális erőforrás felhasználást. A projekt célja ezért egy olyan minimális fájlrendszer megtervezése és implementálása volt, amely megfelel a mérőrendszerek extrém körülményeinek, és garantálja, hogy visszanyerhető legyen az összes rögzített adat.

A dolgozat első részében áttekintettem a háttértárak működését, a fájlrendszerek elméleti és gyakorlati szabályait, a flash memória korlátait és a különböző naplóalapú fájlrendszerek által alkalmazott technikákat. A cél az volt, hogy ezekből a rendszerekből csak azokat az elveket vegyem át, amelyek valóban szükségesek az adott mérési környezetben, és mellőzzek minden olyan funkciót, amely felesleges bonyolultságot vagy erőforrás-igényt okozna.

A fejlesztés során megvalósítottam a ZINF működését: a RAID-1-szerű tükrözési logikát, a szektorokon belüli metaadat-szerkezetet, a CRC32-alapú hibadetektálást, a determinisztikus írási modellt, valamint a metaadatblokk és adatblokk mechanizmusokat, amelyek segítségével a rendszer újraindulás vagy sérült szektorok esetén is képes konzisztensen helyreállni.

A ZINF végül sikeresen teljesítette a kitűzött követelményeket: determinisztikusan működik, robusztus a részlegesen írt vagy sérült szektorokkal szemben, és rendkívül kis erőforrás-igényű, így alkalmas mikrokontrolleres környezetekben történő használatra. A teszteredmények alapján egyértelműen látható, hogy a ZINF – bár rendkívül egyszerű felépítésű – hatékonyan képes helyettesíteni a hagyományos fájlrendszereket olyan mérési szituációkban, ahol a megbízhatóság elsődleges szempont.

A dolgozat lezárásaként kijelenthető, hogy a ZINF megalkotásával sikerült egy olyan minimalista, mégis gyakorlatban jól alkalmazható fájlrendszert létrehozni, amely kifejezetten a CanSat- és CubeSat-rendszerek extrém követelményeihez igazodik, és amely megfelelő alapot jelenthet további kutatásokhoz, optimalizáláshoz vagy kiegészítő funkciók későbbi implementálásához.

12 Köszönetnyilvánítás

Szeretném megköszönni a segítséget először is konzulensemnek, Dr. Várady Gézának, aki nem csak a TDK dolgozatom elkészítése alatt, hanem egyéb projektjeim során is rengeteget támogatott az évek során. Szeretném megköszönni Laborci Gergelynek, Trautman Dávidnak, Tenzlinger Kristófnak, Szajcsán Szabolcsnak, Oláh Gábornak, Sári Dávidnak és Tolnai Máténak is akiktől rengeteg szakmai segítséget és tapasztalatot kaptam mióta együtt dolgozunk. Szeretném még megköszönni családomnak, akik nagyon sokat támogattak tanulmányaiban és lehetőséget adtak arra, hogy tudjak nem csak a TDK-ra, de a szakmai fejlődésre is koncentrálni. És szeretném még megköszönni mindenkinek aki szakmailag vagy egyéb módon támogatott az elmúlt pár évben.

13 Irodalomjegyzék

Hivatkozások

- [1] Dr. Attila Adamkó. *3. fejezet – Fájlok és fájlrendszerek*. URL: <https://gyires.inf.unideb.hu/KMITT/b19/ch03.html>. (Elérve: 2025.11.23).
- [2] System Design School. *File Storage vs. Object Storage – Rendszertervezési áttekintés*. URL: <https://www.youtube.com/watch?v=AV4Ei1qW89o>. (Elérve: 2025.11.23).
- [3] Red Hat. *Fájltárolás, blokk alapú tárolás és objektumtárolás összehasonlítása*. URL: <https://www.redhat.com/en/topics/data-storage/file-block-object-storage>. (Elérve: 2025.11.23).
- [4] IBM. *Objektum-, fájl- és blokk tárolás összehasonlítása*. URL: <https://www.ibm.com/think/topics/object-vs-file-vs-block-storage>. (Elérve: 2025.11.23).
- [5] StoredBits.com. *A naplózó fájlrendszerek megértése: metaadat-, teljes és rendezett journaling*. URL: <https://storedbits.com/journaling-file-systems/>. (Elérve: 2025.11.23).
- [6] linuxbash.sh. *A journaling szerepe a fájlrendszerekben*. URL: <https://www.linuxbash.sh/post/the-role-of-journaling-in-filesystems>. (Elérve: 2025.11.23).
- [7] Caltech. *Journaling File Systems – Előadásanyag*. URL: <https://courses.cms.caltech.edu/cs124/lectures/CS124Lec24.pdf>. (Elérve: 2025.11.23).
- [8] Wikipedia. *ACID tulajdonságok*. URL: <https://hu.wikipedia.org/wiki/ACID>. (Elérve: 2025.11.23).
- [9] Graham Cox. *A BASE modell magyarázata*. URL: <https://www.baeldung.com/cs/db-base-meaning-cap>. (Elérve: 2025.11.23).
- [10] Wikipedia. *A CAP-tétel*. URL: https://en.wikipedia.org/wiki/CAP_theorem. (Elérve: 2025.11.23).
- [11] Keith Tenzer és Joshua Smith. *Idempotencia megértése elosztott rendszerekben*. URL: <https://temporal.io/blog/idempotency-and-durable-execution>. (Elérve: 2025.11.23).
- [12] simplyblock. *A Copy-On-Write (CoW) működése*. URL: <https://www.simplyblock.io/glossary/copy-on-write/>. (Elérve: 2025.11.23).
- [13] F2FS. *F2FS – Flash-Friendly File System dokumentáció*. URL: <https://www.kernel.org/doc/html/latest/filesystems/f2fs.html>. (Elérve: 2025.11.23).

-
- [14] Programming Electronics Academy. *SPIFFS használata ESP32-n – Bevezető útmutató*. URL: <https://www.programmingelectronics.com/spiffs-esp32/>. (Elérve: 2025.11.23).
 - [15] YAFFS. *YAFFS – Yet Another Flash File System*. URL: <https://yaffs.net/>. (Elérve: 2025.11.23).
 - [16] Jan Jongboom. *LittleFS – Nagy integritású beágyazott fájlrendszer*. URL: <https://os.mbed.com/blog/entry/littlefs-high-integrity-embedded-fs/>. (Elérve: 2025.11.23).
 - [17] Prepressure. *RAID*. URL: <https://www.prepressure.com/library/technology/raid>. (Elérve: 2025.11.23).
 - [18] Izsák. *YAML: Mi ez, hogyan működik, és mire használják*. URL: <https://mundobytes.com/hu/yaml-mi-az/>. (Elérve: 2025.11.23).
 - [19] Best Buy. *SD vs SDHC vs SDXC: Which memory card is right for you?* URL: <https://blog.bestbuy.ca/photography/cameras-camcorders/sd-vs-sdhc-vs-sdxc>. (Elérve: 2025.11.23).

Ábrák jegyzéke

1.	Juhász Jenő Szakkollégium által készített Mimike-II Rev-I CanSat Forrás: saját kép	17
2.	European Space Agency, ESEC-Galaxia Education Center, ESA Academy Training Week 2025, Theia ESAT CubeSat Forrás: ESA Academy Training Week 2025, saját kép	18
3.	littlefs (bal) és YAFFS (jobb) logója Forrás: https://yaffs.net/yaffs-2-specification/ , https://avatars.githubusercontent.com/u/60944974?s=48&v=4	21
4.	HDD meghajtó	22
5.	SSD meghajtó	22
6.	Flash memória	23
7.	Multi Média Kártya	23
8.	Tükrök a partíción belül Forrás: Saját kép	29
9.	Tükrök belső struktúrája Forrás: Saját kép	30
10.	Metaadatblokk belső struktúrája Forrás: Saját kép	31
11.	Adatblokk belső struktúrája Forrás: Saját kép	32
12.	C nyelv logója Forrás: https://www.britefish.net/c-programming-language/	34
13.	GNU logója Forrás: https://www.britefish.net/c-programming-language/	35
14.	Tauri (bal) és Svelte (jobb) logója Forrás: https://sveltethemes.dev/auros-one/tauri-sveltekit	37
15.	Késleltetés hisztogram	44

16.	Áteresztőképesség	44
17.	Olvasás visszajelzése Forrás: Saját kép	44
18.	Hibamentes olvasás	45
19.	Alacsony hibainjektálás	45
20.	Közepes hibainjektálás	45
21.	Magas hibainjektálás	45

Táblázatok jegyzéke

1.	A Tauri és az Electron keretrendszerek összehasonlítása	36
----	---	----